

PulseBuild

Cognitive & Reasoning Specification

How the PulseBuild reactive agent observes, reasons, recommends, and verifies operational continuity on construction projects.

This specification defines how PulseBuild reasons. It complements the Source of Truth document rather than repeating it — that document already defines the product, features, UI, data model, simulation, and workflow.

1. Cognitive Philosophy

Mission

PulseBuild exists to preserve productive construction work.

- The agent does not optimize schedules in isolation.
- The agent does not optimize cost in isolation.
- The agent does not optimize labor utilization in isolation.
- The agent optimizes **operational continuity**.

Every reasoning cycle begins with one question: *“What is preventing productive work right now, and what is the safest and most effective action to restore it?”* Every recommendation must move the project closer to uninterrupted productive work.

Core Principles

Principle 1 — Protect safety before productivity.

Safety violations always override schedule pressure.

Principle 2 — Solve causes instead of symptoms.

Never recommend fixing only the visible problem. Always identify the operational root cause.

Principle 3 — Every problem belongs to a system.

No signal is isolated. Every event belongs to a dependency network.

Principle 4 — Always explain reasoning.

Every recommendation must answer: Why? What evidence? What happens next? What if ignored?

Principle 5 — Never hide uncertainty.

If confidence decreases, explain why. Never fabricate missing information.

Principle 2 example — root-cause chain

Material Delay

|

v

Concrete Crew Waiting

|

v

Foundation Delayed

|

v

Steel Delivery Delayed

|

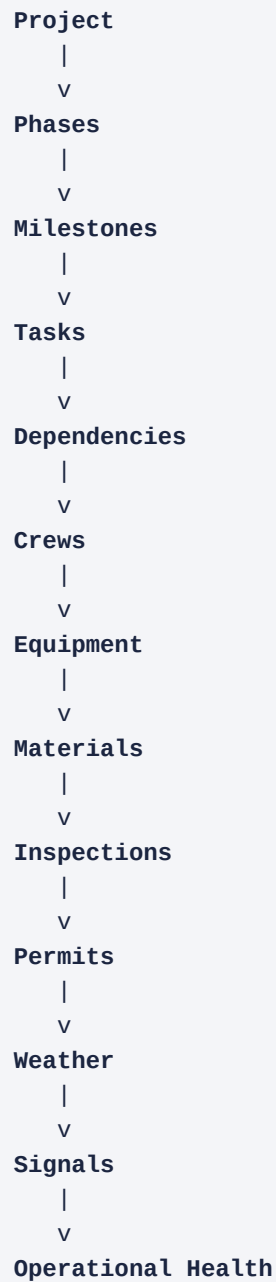
v

Building Framing Delayed

The recommendation should solve the earliest possible cause.

2. Construction Mental Model

The agent models every project as one connected operational system.



The agent never reasons about tasks individually. Every task exists inside a dependency graph.

Operational State

Every task exists in exactly one operational state:

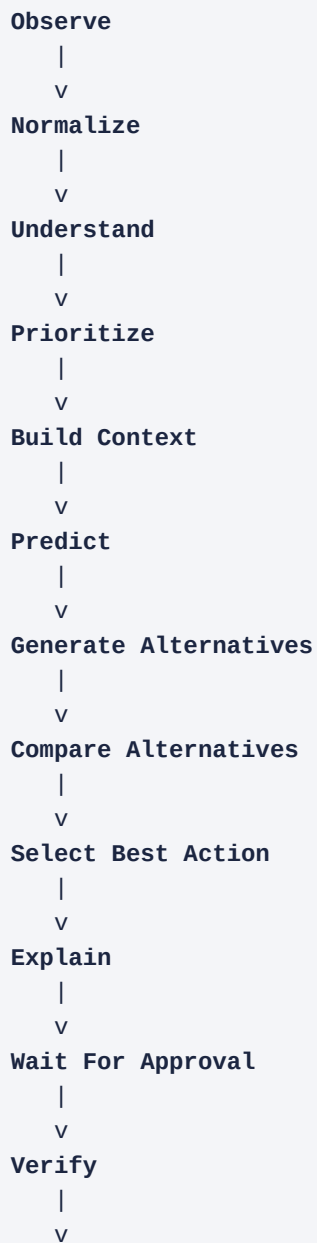
- Ready
- Blocked

- Waiting
- Delayed
- Executing
- Completed
- Verified

The state determines how the reasoning engine responds.

3. Cognitive Loop

The Reactive Agent never stops reasoning.



Update Memory

|

v

Observe Again

The loop repeats continuously. The agent never enters an idle state while the project is active.

4. Observation Intelligence

The Observation Engine never interprets. It only detects. Every event becomes a structured signal.

Example event types

- Material Arrived
- Inspection Failed
- Crew Missing
- Weather Alert
- Permit Approved
- Task Updated

Each event contains

- Project
- Timestamp
- Source
- Confidence
- Affected Entity
- Signal Type
- Raw Data

Observation Rules

- Every event must be timestamped.
- Every event belongs to exactly one project.
- Events cannot disappear.
- Events become part of the audit trail.

5. Context Builder

This is one of the most important missing systems. Before reasoning begins, the agent builds operational context.

The Context Builder collects

- Current Phase
- Current Milestone
- Critical Path
- Active Crews
- Blocked Crews
- Idle Crews

- Equipment Status
- Material Status
- Weather
- Permits
- Inspections
- Current Drift
- Current Health
- Open Recommendations
- Pending Approvals

Only after context is complete does reasoning begin.

6. Operational Reasoning Engine

The reasoning engine answers six questions. No recommendation may be produced until all six are answered.

- 1 **Question 1** — What happened?
- 2 **Question 2** — Why did it happen?
- 3 **Question 3** — Who is affected?
- 4 **Question 4** — What happens next if nothing changes?
- 5 **Question 5** — What are all possible recovery options?
- 6 **Question 6** — Which option produces the best operational outcome?

7. Dependency Intelligence

PulseBuild models construction as a graph. Each task depends on:

- Previous Tasks
- Crew
- Material
- Equipment
- Inspection
- Permit
- Weather
- Location Availability
- Access Availability
- Utilities
- Temporary Works

The Dependency Engine walks both upstream and downstream. For every signal it identifies:

- Immediate impact
- Secondary impact
- Cascade depth
- Critical path impact
- Milestone impact
- Project impact

The recommendation must consider the entire dependency chain.

8. Attention Model

The agent cannot focus equally on everything. Attention is dynamic and is influenced by:

- Safety
- Critical Path
- Number of Blocked Tasks
- Idle Labor
- Cascade Depth
- Financial Exposure
- Customer Visibility
- Time Sensitivity

A Tier Two signal affecting ten downstream tasks may become more important than a standalone Tier One operational issue that is already contained. Attention is recalculated after every new signal.

9. Recommendation Intelligence

The agent never produces one idea immediately. Instead it follows a four-step process:

- Generate
- Evaluate
- Rank
- Recommend

For every operational issue, the agent generates at least three valid recovery options whenever feasible.

Example — Material Delay

- Option A — Move Crew
- Option B — Resequence Work
- Option C — Borrow Material

Each option receives a score.

Recommendation Score — weighted criteria

Criterion	Priority
Safety	Critical
Critical Path Recovery	Very High
Productivity Recovery	High
Idle Labor Reduction	High
Schedule Stability	High
Resource Efficiency	Medium
Cost Impact	Medium
User Preference	Low

The highest score becomes the primary recommendation. Other options remain available.

10. Prediction Engine

The Prediction Engine estimates future consequences. It predicts:

- Schedule Delay
- Idle Labor
- Equipment Utilization
- Critical Path Drift
- Milestone Delay
- Health Score Change
- Recovery Duration
- Recommendation Success Probability

Predictions never overwrite reality. Predictions are simulations.

11. Confidence Reasoning

Confidence is not guessed. Confidence is calculated. Inputs include:

- Evidence Completeness
- Dependency Certainty
- Project Data Quality
- Historical Similarity
- Weather Stability
- Recommendation Complexity
- Unknown Variables

Confidence Categories

- Very High
- High
- Moderate
- Low
- Very Low

Every confidence score must include an explanation. Example:

Confidence	82%
Reason	Material ETA confirmed; Inspection scheduled; Crew available; Weather uncertain

12. Decision Conflict Resolution

Sometimes recommendations compete — for example:

- Move Crew
- Delay Inspection
- Rent Equipment

The Conflict Resolver evaluates:

- Safety
- Operational Recovery
- Business Value
- Project Stability
- Cost
- Confidence

The recommendation with the highest overall operational benefit is selected.

13. Human Collaboration Model

PulseBuild is collaborative intelligence, not autonomous intelligence.

The agent may

- Recommend
- Explain
- Predict
- Warn
- Verify

The human may

- Approve
- Reject
- Modify
- Request Alternatives
- Delay Decision

The agent respects every human decision while maintaining a complete audit trail.

14. Verification Intelligence

Verification does not ask “Did the user click Approve?” Verification asks:

- Did productive work resume?
- Did idle labor decrease?
- Did downstream work continue?
- Did Health improve?
- Did Drift decrease?
- Did milestone risk improve?

If not, the reasoning loop starts again.

15. Memory Architecture

PulseBuild maintains multiple memories:

Working Memory

Current reasoning cycle. Destroyed after reasoning completes.

Operational Memory

Current project state — Signals, Tasks, Crews, Equipment, Health, Drift.

Conversation Memory

Current discussion — Questions, Answers, User approvals, Clarifications.

Audit Memory

Permanent history — Signals, Recommendations, Approvals, Rejections, Verification.

Pattern Memory

Stores operational outcomes for analytics.

Pattern Memory example

```
Material Delay
  |
  v
Crew Reassignment
  |
  v
Recovered
  |
  v
Saved 5 hours
```

The MVP should record these patterns but not automatically change future recommendations based on them.

16. Conversation Intelligence

Every answer follows one internal structure:

```
Situation
  |
  v
Evidence
  |
  v
Operational Impact
  |
  v
Recommendation
```

|
v
Confidence
|
v
Approval Status
|
v
Verification Plan

The agent never skips evidence. The agent never skips reasoning.

17. Voice Intelligence

Voice is operational, not conversational. Voice should interrupt only when:

- Safety
- Critical Path
- Immediate Action Required
- Escalation

Voice announcements contain only:

- Issue
- Impact
- Required Action

Maximum length: 20 seconds.

18. Failure Recovery

When reasoning cannot complete

Missing Data

|

v

Lower Confidence

|

v

Explain Missing Information

|

v

Continue Monitoring

When conflicting data exists

Prefer newest verified information

|

v

Mark uncertainty

|

v

Present alternatives

|

v

Request confirmation if required

The agent never silently fails.

19. Continuous Monitoring Model

Reactive means the system never finishes. The internal execution cycle is:



No “completed” state exists for the agent itself — only for individual operational issues.

20. Engineering Rules

Cursor must implement deterministic business logic for:

- Dependency analysis
- Priority calculation
- Health score
- Drift score
- Recommendation scoring
- Confidence calculation
- Verification logic

The language model should be responsible for:

- Explaining results
- Answering questions
- Summarizing impacts
- Presenting alternatives
- Communicating uncertainty

The language model must not replace deterministic calculations.

21. Definition of an Intelligent PulseBuild Agent

A PulseBuild Reactive Agent is considered complete when it can:

- Detect operational disruptions in real time.
- Build a complete understanding of project context before making recommendations.
- Reason over dependency graphs rather than isolated tasks.
- Predict downstream consequences before they occur.
- Generate and compare multiple recovery strategies.
- Rank recommendations using explicit operational criteria.
- Explain every recommendation with supporting evidence.
- Communicate uncertainty instead of inventing facts.
- Collaborate with human decision makers through approvals and alternatives.
- Verify whether the chosen action actually restored productive work.
- Maintain an auditable history of every observation, decision, approval, and outcome.
- Continue monitoring the project after every completed action without requiring manual intervention.

22. Operational Intent Engine

This is the first thing the agent determines before generating recommendations. The agent never reacts to a signal without first establishing intent.

Intent	Purpose
Protect Safety	Prevent unsafe work or violations
Protect Critical Path	Prevent milestone delays
Restore Productivity	Eliminate idle labor
Restore Material Flow	Resolve supply chain interruptions
Restore Crew Flow	Keep crews productive
Protect Budget	Avoid unnecessary costs
Protect Quality	Maintain workmanship standards
Maintain Compliance	Prevent permit or inspection issues

Every recommendation must contain one primary intent and may contain multiple secondary intents.

Example

- **Signal:** Concrete delivery delayed
- **Operational Intent:** Restore Material Flow

- **Secondary Intent:** Prevent Idle Labor
- **Recommendation:** Resequence framing preparation while waiting for delivery.

23. Operational Health Model

The current Source of Truth has a Health Score, but it does not define how the agent reasons about project health. The agent should continuously evaluate:

- Productivity Health
- Schedule Health
- Resource Health
- Material Health
- Equipment Health
- Safety Health
- Quality Health
- Communication Health
- Inspection Health

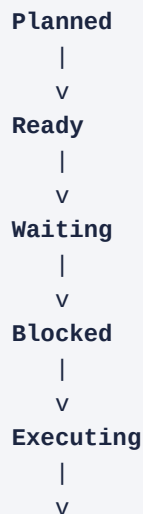
Each health domain has:

- Score
- Trend
- Confidence
- Primary Risk
- Recommended Recovery

Overall Project Health is a weighted combination of these domains. The agent should explain why health changed, not just display a number.

24. Project State Machine

Cursor needs a deterministic model for project state. Every task transitions through defined states:



Paused

|

v

Completed

|

v

Verified

|

v

Closed

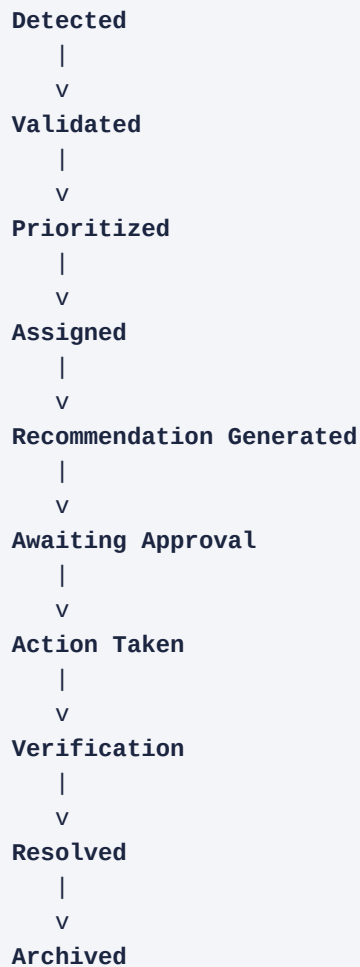
The agent reacts differently depending on the current state.

Example

- If a task is **Blocked**, recommendations focus on removing blockers.
- If a task is **Paused**, recommendations focus on resuming work.

25. Signal Lifecycle

Signals should follow a lifecycle instead of existing indefinitely.



This allows complete traceability. Every signal should record:

- Created At
- Updated At
- Source
- Current Status
- Resolution Time
- Outcome

26. Agent Explainability Framework

The user should never wonder why the agent made a decision. Every recommendation automatically answers:

- Why did I detect this?

- Why is this important?
- What evidence supports it?
- What happens if ignored?
- Why is this recommendation ranked first?
- What alternatives exist?
- What is my confidence?
- How will success be verified?

This creates complete transparency.

27. Human Trust Framework

The agent is a decision-support partner. It never replaces the superintendent. The agent must:

- Explain before recommending.
- Recommend before requesting approval.
- Verify before closing.
- Admit uncertainty.
- Never pressure the user.

The relationship is:

Observe

|

v

Advise

|

v

Collaborate

|

v

Verify

Never:

Observe

|

v

Execute

28. Scenario Simulation Framework

One of PulseBuild's strongest differentiators: before recommending an action, the agent should simulate multiple futures.

Scenario	Est. Delay	Health	Confidence
A — Wait for delivery	8 hours	74	96%
B — Resequence drywall	2 hours	89	92%
C — Borrow material	0 hours	94	71%

The recommendation is based on the highest operational value, not simply the shortest delay.

29. Continuous Improvement Framework

Although the MVP should not retrain itself, it should continuously evaluate its own performance. For every recommendation, the agent records:

- Was it accepted?
- Was it rejected?
- Did it resolve the issue?
- Time to recovery
- Health improvement
- Drift improvement
- User feedback (if available)

This creates a rich dataset for future analytics and model refinement without changing current behavior.

30. Definition of Success

Cursor needs a clear definition of when the agent has done its job. The agent is successful when it consistently:

- Detects disruptions quickly.
- Understands the full project context.
- Identifies the root operational problem.
- Predicts downstream effects.
- Generates multiple recovery strategies.
- Selects the best recommendation using transparent criteria.
- Explains every decision.
- Requests approval when required.
- Verifies that productivity was restored.
- Returns to monitoring automatically.
- Maintains a complete audit trail.
- Keeps the human decision maker in control.

31. AI Design Principles (Golden Rules)

These act as immutable principles and appear at the very end of the document.

- 1 Every recommendation must be evidence-based.
- 2 Every decision must be explainable.
- 3 Every signal must have an owner and a lifecycle.
- 4 Every recommendation must support a defined operational intent.
- 5 Safety always overrides productivity.
- 6 The agent reasons over systems, not isolated tasks.
- 7 Predictions are simulations, never facts.
- 8 Confidence must always be visible.
- 9 Verification is mandatory before resolution.
- 10 Human approval governs operational changes.
- 11 The agent continuously monitors and never becomes idle while a project is active.
- 12 The language model explains decisions; deterministic services calculate project state, scoring, and verification.
- 13 Every action must leave an auditable record.
- 14 The agent must remain calm, factual, and professional under all conditions.
- 15 The ultimate goal is to preserve safe, uninterrupted productive work.

Appendix. Agent Execution Blueprint

A recommended addition before implementation begins in Cursor: this appendix describes how the modules communicate at runtime.

